

Part B – match the mock-up

The dialog works but does not look like the specification. Five steps: OK and Cancel instead of Close, the button moved onto the preference page in place of Restore Defaults, and the composite laid out as two tables side by side. Client rebuild only, no redeploy.

The mock-up shows	The build shows	Fixed by
Two tables side by side: law names left, that law's parameters right with Contextual parameter, Min Value, Max Value and Stability columns	One table with the parameters squeezed into a single text column	Step 4
Filtering law name box on the left, CTXT1 to CTXT5 rows on the right	Name above, rows below	Step 4
Create Filtering Law, Update Filtering Law, Delete Filtering Law	Create, Update, Delete	Step 5
OK and Cancel	Close, showing !Missing i18n key Close!	Step 1
Manage Filtering Laws where Restore Defaults sits	An extra button inside the page	Steps 2 and 3

The five steps

Step 1 — FilteringLawDialog – use the standard buttons

mgs-health-app/.../preferences/healthalarms/FilteringLawDialog.java

Delete both of these methods from the class. Nothing replaces them: with the overrides gone, JFace draws OK and Cancel itself with its own localised labels, which is what the mock-up shows and what removes the missing key message.

```
// DELETE
@Override
protected void createButtonsForButtonBar(Composite parent) {
    toolkit.adapt(parent);
    createButton(parent, IDialogConstants.CLOSE_ID,
        Messages.CORE.getMnemonicEscapedString("Close"), true);
    ColorUtils.forceBackgroundColor(parent);
}

// DELETE
@Override
protected void buttonPressed(int buttonId) {
    if (buttonId == IDialogConstants.CLOSE_ID) {
        close();
    }
}
```

Keep the constructor, configureShell and createDialogArea exactly as they are. Eclipse will flag IDialogConstants and ColorUtils as unused – press Ctrl+Shift+O to clean the imports.

Create, Update and Delete act immediately, so OK and Cancel both just close the dialog. That matches the mock-up. If the client later wants Cancel to undo the changes made while the dialog was open, that is a different design and worth raising then.

Step 2 — HealthAlarmsComposite – button out, method in

mgs-health-app/.../preferences/healthalarms/HealthAlarmsComposite.java

Two edits.

2.1 Delete the whole block you added in the previous step 13, in the constructor after the tabs:

```
// DELETE all of this
Button manageLawsButton = new Button(this, SWT.PUSH);
manageLawsButton.setText(Messages.CORE.getString("filteringLawManageButton"));
manageLawsButton.setLayoutData(new GridData(SWT.END, SWT.CENTER, true, false));
manageLawsButton.addListener(SWT.Selection, e -> {
    if (selectedAircraft != null) {
        new FilteringLawDialog(healthAlarmsPreferenceStore, selectedAircraft).open();
        reloadAll();
    }
});
```

2.2 Add this public method, directly under reloadAll(). The page calls it in Step 3.

```
/**
 * BCR-6605 open the Manage Filtering Laws dialog for the selected aircraft.
 */
public void openFilteringLawDialog() {
    if (selectedAircraft != null) {
        new FilteringLawDialog(healthAlarmsPreferenceStore, selectedAircraft).open();
        reloadAll();
    }
}
```

The import of Button can go if nothing else in the class uses it.

Step 3 — HealthAlarmsPreferencePage – swap Restore Defaults for the button

mgs-health-app/.../preferences/healthalarms/HealthAlarmsPreferencePage.java

Two edits. The composite field in this class is called hac.

3.1 Add one import:

```
import org.eclipse.swt.widgets.Button;
```

3.2 Add both methods, directly after createContents:

```
@Override
protected void contributeButtons(Composite parent) {
    ((GridLayout) parent.getLayout()).numColumns++;
    Button manageLaws = new Button(parent, SWT.PUSH);
    manageLaws.setText(Messages.CORE.getString("filteringLawManageButton"));
    manageLaws.addListener(SWT.Selection, e -> hac.openFilteringLawDialog());
}

@Override
public void createControl(Composite parent) {
    super.createControl(parent);
    Button defaults = getDefaultsButton();
    if (defaults != null) {
        defaults.setVisible(false);
    }
}
```

Nothing else changes. SWT, GridLayout, Composite and Messages are already imported, and the class has no createControl today, so there is nothing to merge.

The existing performDefaults() override still calls hac.performDefaults(). With the button hidden nothing can trigger it, so it becomes dead code – harmless, but add a short comment so the next reader does not spend time wondering why it never runs.

Hiding rather than removing keeps the layout stable: the button still takes its slot, so Apply and Apply and Close stay where the user expects them.

Step 4 — FilteringLawComposite – replace the whole class

mgs-health-app/.../preferences/healthalarms/FilteringLawComposite.java

Select all in the file and paste this over it. The logic is unchanged from the version that works; only the layout and one extra table are new.

What changed, in case you are comparing: the single master table became two tables in a two-column composite; the detail area became name on the left and CTXT rows on the right; a ParameterLabelProvider was added for the new table; and showLaw now also sets that table's input.

```
package com.eurocopter.mgs.presentation.configuration.preferences.healthalarms;

import java.text.MessageFormat;
import java.util.ArrayList;
import java.util.Arrays;
import java.util.List;

import org.eclipse.jface.dialogs.MessageDialog;
import org.eclipse.jface.viewers.ArrayContentProvider;
import org.eclipse.jface.viewers.IStructuredSelection;
import org.eclipse.jface.viewers.ITableLabelProvider;
import org.eclipse.jface.viewers.LabelProvider;
import org.eclipse.jface.viewers.TableViewer;
import org.eclipse.swt.SWT;
import org.eclipse.swt.graphics.Image;
import org.eclipse.swt.layout.GridData;
import org.eclipse.swt.layout.GridLayout;
import org.eclipse.swt.widgets.Button;
import org.eclipse.swt.widgets.Combo;
import org.eclipse.swt.widgets.Composite;
import org.eclipse.swt.widgets.Label;
import org.eclipse.swt.widgets.Table;
import org.eclipse.swt.widgets.TableColumn;
import org.eclipse.swt.widgets.Text;

import com.eurocopter.mgs.core.presentation.i18n.Messages;
import com.eurocopter.mgs.datamodel.aircraft.Aircraft;
import com.eurocopter.mgs.presentation.configuration.preferences.healthalarms.store.HealthAlarmsPreferenceStore;
import com.eurocopter.mgs.service.health.common.dto.ContextualParameterDto;
import com.eurocopter.mgs.service.health.common.dto.FilteringLawDto;

public class FilteringLawComposite extends Composite {

    private static final int MAX_PARAMETERS = 5;

    private final HealthAlarmsPreferenceStore store;
    private final Aircraft aircraft;

    private final TableViewer lawViewer;
    private final TableViewer parameterViewer;
    private final Text nameText;
    private final Combo[] parameterCombos = new Combo[MAX_PARAMETERS];
    private final Text[] minTexts = new Text[MAX_PARAMETERS];
    private final Text[] maxTexts = new Text[MAX_PARAMETERS];
    private final Text[] stabilityTexts = new Text[MAX_PARAMETERS];
    private final Button createButton;
    private final Button updateButton;
    private final Button deleteButton;

    private FilteringLawDto selectedLaw;

    public FilteringLawComposite(Composite parent, HealthAlarmsPreferenceStore store,
        Aircraft aircraft) {
        super(parent, SWT.NONE);
        this.store = store;
        this.aircraft = aircraft;
        setLayout(new GridLayout(1, false));
```

```

// ----- top: laws on the left, the selected law's parameters on the right
Composite top = new Composite(this, SWT.NONE);
top.setLayout(new GridLayout(2, false));
GridData topData = new GridData(SWT.FILL, SWT.FILL, true, true);
topData.heightHint = 200;
top.setLayoutData(topData);

Table lawTable = new Table(top, SWT.BORDER | SWT.FULL_SELECTION | SWT.V_SCROLL);
lawTable.setHeaderVisible(true);
lawTable.setLinesVisible(true);
GridData lawData = new GridData(SWT.FILL, SWT.FILL, false, true);
lawData.widthHint = 260;
lawTable.setLayoutData(lawData);
TableColumn lawColumn = new TableColumn(lawTable, SWT.LEFT);
lawColumn.setText(Messages.CORE.getString("filteringLawColLaw"));
lawColumn.setWidth(250);

lawViewer = new TableViewer(lawTable);
lawViewer.setContentProvider(ArrayContentProvider.getInstance());
lawViewer.setLabelProvider(new LawLabelProvider());
lawViewer.addSelectionChangedListener(event -> {
    IStructuredSelection selection = (IStructuredSelection) lawViewer.getSelection();
    showLaw((FilteringLawDto) selection.getFirstElement());
});

Table parameterTable = new Table(top, SWT.BORDER | SWT.FULL_SELECTION | SWT.V_SCROLL);
parameterTable.setHeaderVisible(true);
parameterTable.setLinesVisible(true);
parameterTable.setLayoutData(new GridData(SWT.FILL, SWT.FILL, true, true));
createParameterColumns(parameterTable);

parameterViewer = new TableViewer(parameterTable);
parameterViewer.setContentProvider(ArrayContentProvider.getInstance());
parameterViewer.setLabelProvider(new ParameterLabelProvider());

// ----- bottom: name on the left, CTXT rows on the right
Composite bottom = new Composite(this, SWT.NONE);
bottom.setLayout(new GridLayout(2, false));
bottom.setLayoutData(new GridData(SWT.FILL, SWT.TOP, true, false));

Composite namePart = new Composite(bottom, SWT.NONE);
namePart.setLayout(new GridLayout(1, false));
GridData nameData = new GridData(SWT.FILL, SWT.TOP, false, false);
nameData.widthHint = 260;
namePart.setLayoutData(nameData);
new Label(namePart, SWT.NONE).setText(Messages.CORE.getString("filteringLawNameLabel"));
nameText = new Text(namePart, SWT.BORDER);
nameText.setLayoutData(new GridData(SWT.FILL, SWT.CENTER, true, false));

Composite rows = new Composite(bottom, SWT.NONE);
rows.setLayout(new GridLayout(5, false));
rows.setLayoutData(new GridData(SWT.FILL, SWT.TOP, true, false));

new Label(rows, SWT.NONE).setText("");
new Label(rows, SWT.NONE).setText(Messages.CORE.getString("filteringLawColContextual"));
new Label(rows, SWT.NONE).setText(Messages.CORE.getString("filteringLawColMin"));
new Label(rows, SWT.NONE).setText(Messages.CORE.getString("filteringLawColMax"));
new Label(rows, SWT.NONE).setText(Messages.CORE.getString("filteringLawColStability"));

```

```

String[] parameterNames = parameterNames();
for (int i = 0; i < MAX_PARAMETERS; i++) {
    new Label(rows, SWT.NONE).setText("CTXT" + (i + 1));
    parameterCombos[i] = new Combo(rows, SWT.READ_ONLY | SWT.BORDER);
    parameterCombos[i].setItems(parameterNames);
    GridData comboData = new GridData(SWT.FILL, SWT.CENTER, true, false);
    comboData.widthHint = 230;
    parameterCombos[i].setLayoutData(comboData);
    minTexts[i] = numberField(rows);
    maxTexts[i] = numberField(rows);
    stabilityTexts[i] = numberField(rows);
}

// ----- buttons
Composite buttons = new Composite(this, SWT.NONE);
buttons.setLayout(new GridLayout(3, false));
buttons.setLayoutData(new GridData(SWT.CENTER, SWT.CENTER, true, false));
createButton = new Button(buttons, SWT.PUSH);
createButton.setText(Messages.CORE.getString("filteringLawCreate"));
createButton.addListener(SWT.Selection, e -> create());
updateButton = new Button(buttons, SWT.PUSH);
updateButton.setText(Messages.CORE.getString("filteringLawUpdate"));
updateButton.addListener(SWT.Selection, e -> update());
deleteButton = new Button(buttons, SWT.PUSH);
deleteButton.setText(Messages.CORE.getString("filteringLawDelete"));
deleteButton.addListener(SWT.Selection, e -> delete());

reload();
showLaw(null);
}

private void createParameterColumns(Table table) {
    String[] keys = { "filteringLawColContextual", "filteringLawColMin",
        "filteringLawColMax", "filteringLawColStability" };
    int[] widths = { 240, 90, 90, 90 };
    for (int i = 0; i < keys.length; i++) {
        TableColumn column = new TableColumn(table, SWT.LEFT);
        column.setText(Messages.CORE.getString(keys[i]));
        column.setWidth(widths[i]);
    }
}

private Text numberField(Composite parent) {
    Text field = new Text(parent, SWT.BORDER);
    GridData data = new GridData(SWT.FILL, SWT.CENTER, false, false);
    data.widthHint = 80;
    field.setLayoutData(data);
    return field;
}

/** The parameter dropdown comes straight from the enum. */
private String[] parameterNames() {
    com.eurocopter.mgs.datamodel.health.common.ContextualParameter[] values =
        com.eurocopter.mgs.datamodel.health.common.ContextualParameter.values();
    String[] names = new String[values.length];
    for (int i = 0; i < values.length; i++) {
        names[i] = values[i].name();
    }
}

```

```

    }
    Arrays.sort(names);
    return names;
}

private void reload() {
    lawViewer.setInput(store.getFilteringLawDetails(aircraft));
}

private void showLaw(FilteringLawDto law) {
    selectedLaw = law;
    boolean readOnly = law != null && (law.isOriginal() || law.isInUse());

    parameterViewer.setInput(law != null ? law.getParameters() : new ArrayList<>());

    nameText.setText(law != null && law.getName() != null ? law.getName() : "");
    for (int i = 0; i < MAX_PARAMETERS; i++) {
        ContextualParameterDto p = law != null && i < law.getParameters().size()
            ? law.getParameters().get(i) : null;
        parameterCombos[i].setText(p != null && p.getName() != null ? p.getName() : "");
        minTexts[i].setText(p != null ? text(p.getMinValue()) : "");
        maxTexts[i].setText(p != null ? text(p.getMaxValue()) : "");
        stabilityTexts[i].setText(p != null ? text(p.getThresholdOfVariabilityValue()) : "");
    }
    setFormEnabled(!readOnly);
    updateButton.setEnabled(law != null && !readOnly);
    deleteButton.setEnabled(law != null && !readOnly);
}

private void setFormEnabled(boolean enabled) {
    nameText.setEnabled(enabled);
    for (int i = 0; i < MAX_PARAMETERS; i++) {
        parameterCombos[i].setEnabled(enabled);
        minTexts[i].setEnabled(enabled);
        maxTexts[i].setEnabled(enabled);
        stabilityTexts[i].setEnabled(enabled);
    }
}

private String text(Double value) {
    return value == null ? "" : String.valueOf(value);
}

private Double value(Text field) {
    String raw = field.getText().trim();
    if (raw.isEmpty()) {
        return null;
    }
    try {
        return Double.valueOf(raw);
    } catch (NumberFormatException e) {
        return null;
    }
}

private FilteringLawDto readForm(String id) {
    String name = nameText.getText().trim();
    if (name.isEmpty()) {

```

```

        error("filteringLawErrorName");
        return null;
    }
    for (Object existing : (List<?>) lawViewer.getInput()) {
        FilteringLawDto other = (FilteringLawDto) existing;
        if (name.equals(other.getName()) && !other.getId().equals(id)) {
            error("filteringLawErrorDuplicate");
            return null;
        }
    }

    FilteringLawDto dto = new FilteringLawDto();
    dto.setId(id);
    dto.setName(name);
    for (int i = 0; i < MAX_PARAMETERS; i++) {
        String parameter = parameterCombos[i].getText().trim();
        if (parameter.isEmpty()) {
            continue; // this row is simply unused
        }
        Double min = value(minTexts[i]);
        Double max = value(maxTexts[i]);
        if (min == null || max == null || min >= max) {
            error("filteringLawErrorMinMax");
            return null;
        }
        ContextualParameterDto p = new ContextualParameterDto();
        p.setName(parameter);
        p.setMinValue(min);
        p.setMaxValue(max);
        p.setThresholdOfVariabilityValue(value(stabilityTexts[i]));
        dto.getParameters().add(p);
    }
    if (dto.getParameters().isEmpty()) {
        error("filteringLawErrorNoParameter");
        return null;
    }
    return dto;
}

private void error(String key) {
    MessageDialog.openError(getShell(),
        Messages.CORE.getString("filteringLawErrorTitle"),
        Messages.CORE.getString(key));
}

private void create() {
    FilteringLawDto dto = readForm(null);
    if (dto == null) {
        return;
    }
    store.createFilteringLaw(aircraft, dto);
    reload();
    showLaw(null);
}

private void update() {
    if (selectedLaw == null) {
        return;
    }
}

```

```

    }
    FilteringLawDto dto = readForm(selectedLaw.getId());
    if (dto == null) {
        return;
    }
    store.updateFilteringLaw(dto);
    reload();
    showLaw(null);
}

private void delete() {
    if (selectedLaw == null) {
        return;
    }
    boolean confirmed = MessageDialog.openConfirm(getShell(),
        Messages.CORE.getString("filteringLawDeleteTitle"),
        MessageFormat.format(Messages.CORE.getString("filteringLawDeleteConfirm"),
            selectedLaw.getName()));
    if (!confirmed) {
        return;
    }
    store.deleteFilteringLaw(aircraft, selectedLaw.getId());
    reload();
    showLaw(null);
}

private final class LawLabelProvider extends LabelProvider implements ITableLabelProvider {

    @Override
    public Image getColumnImage(Object element, int column) {
        return null;
    }

    @Override
    public String getColumnText(Object element, int column) {
        return ((FilteringLawDto) element).getName();
    }
}

private final class ParameterLabelProvider extends LabelProvider
    implements ITableLabelProvider {

    @Override
    public Image getColumnImage(Object element, int column) {
        return null;
    }

    @Override
    public String getColumnText(Object element, int column) {
        ContextualParameterDto p = (ContextualParameterDto) element;
        switch (column) {
            case 0: return p.getName();
            case 1: return text(p.getMinValue());
            case 2: return text(p.getMaxValue());
            case 3: return text(p.getThresholdOfVariabilityValue());
            default: return "";
        }
    }
}
}
}

```

Step 5 — CoreMessages.properties – and every language file

Three keys change their text, five are new, two are no longer used.

5.1 Change the text of these three, so the buttons read as the mock-up does:

```

filteringLawCreate=Create Filtering Law
filteringLawUpdate=Update Filtering Law
filteringLawDelete=Delete Filtering Law

```

5.2 Add these five:

```
filteringLawColLaw=Filtering law
filteringLawColContextual=Contextual parameter
filteringLawColMin=Min Value
filteringLawColMax=Max Value
filteringLawNameLabel=Filtering law name
```

5.3 Delete these two, nothing uses them now:

```
filteringLawColName
filteringLawColParameter
```

filteringLawColStability, filteringLawManageButton, filteringLawDialogTitle, the three delete keys and the five error keys all stay as they are.

Build and test

#	Action
1	Rebuild the client only. No server change in this set, so no redeploy.
2	Open Health, pick an aircraft – the button row now reads Manage Filtering Laws, Apply, and Restore Defaults is gone
3	Press it – the dialog opens with laws on the left and an empty parameter table on the right
4	Select a law – its parameters fill the right table, one row each, and the CTXT rows below fill too
5	Select a law already used by an indicator – the form greys out, Update and Delete disable, but the right table still shows its parameters
6	Create a law with two parameters – it appears on the left, and selecting it shows two rows on the right
7	The bottom of the dialog reads OK and Cancel, with no missing key message

If Restore Defaults is still visible after item 2, createControl is not being called the way Step 3 assumes – send me HealthAlarmsPreferencePage.java and I will place it against the real class.